Architectural Decisions

Table of Contents

- 1. Overall state transition
  - 1.1. 1.1 Definition
  - 1.2. Design decisions
  - 1.3. Visual description
- 2. Local vaults
  - 2.1. Vaults as smart contracts
  - 2.2. Limited responsibility
  - 2.3. Local transportation to/from wallets
  - 2.4. Local moves of Staking Stock
- 3. Omnichain staking
  - 3.1. Definition
  - 3.2. Regular asset move plans
  - 3.3. Design decisions
  - 3.4. Transitioning staking
- 4. Omnichain mLP token
  - 4.1. Requirements analysis
  - 4.2. Design decisions
- 5. Logical components
- 6. Omnichain vault
- 7. Staking planner
  - 7.1. Task definition
  - 7.2. Definition
  - 7.3. Formula
- 8. Inter-chain transportation
  - 8.1. Considerations
  - 8.2. Decentralized operations required
  - 8.3. Employ the LayerZero service
  - 8.4. Operations exemptible of decentralization
  - 8.5. Design recommendations
  - 8.6. An off-chain detour for inter-chain transportation
- 9. Miscellaneous
  - 9.1. Compounding
    - 9.1.1. Considerations
  - 9.2. Design recommendations
  - 9.3. Gas supply
    - 9.3.1. Considerations
    - 9.3.2. Design recommendations
  - 9.4. Auxiliary descriptions of the architecture
    - 9.4.1. Deposit / Withdraw - deposits and rewards mixed 1:1
- 10. Reference source code

Architectural Decisions

- Definition
  - Mozaic, the system, and Mozaic system refers to the software system that this project is going to develop, launch, and operate.
- Goal
  - This document describes architectural decisions that implement the system requirements.
  - This document serves as additional technical terminology of the project.

1. Overall state transition

1.1. 1.1 Definition

- **System Asset Snapshot**, **system asset/request state**, or simply **asset state**, is the state at a given time and identified by the followings:

  - Note:
    - The following pseudo-code is to describe concepts consciously and not the implementation code.
    - Different fields of the following structures are calculated at different states.

  - **Deposits**: All deposit requests.
    A deposit request can be modeled by:

```
struct Deposit {    // sends assets to the system and receives mLP in return
    address user;       // the user who requests the deposit
    address token;      // the denominator token of the assets
    uint    token_chain // the chain that hosts the denominator token
    uint    amount;     // the amount of the assets in the denominator token
    uint    amountLP;   // the amount of mLP
    uint    lp_chain;   // the chain that hosts the mLP token
    uint    usdt_equ;   // the usdt-equivalent of the asset amount
}
```

  - **Withdrawals**: All withdrawal requests. A withdrawal request can be modeled by:

```
struct Withdraw {   // sends mLP to the system and receives assets in return
    address user;       // the user who requests the withdrawal
    address token;      // the denominator token of the assets
    uint    token_chain // the chain that hosts the denominator token
    uint    amount;     // the amount of the assets in the denominator token
    uint    amountLP;   // the amount of mLP
    uint    lp_chain;   // the chain that hosts the mLP token
    uint    usdt_equ;   // the usdt-equivalent of the asset amount
}
```

- **Stakes**: All staked chunks. A staked chunk can be modeled by:

```
struct Stake {   // a stake of asset is staked on a pool
    address token;      // the denominator token of the assets
    uint    token_chain // the chain that hosts the denominator token
    uint    amount;     // the asset amount in the denominator token
    uint    pool_id;    // the local/global pool ID
    uint    usdt_equ;   // the usdt-equivalent of the asset amount
}
```

- **Rewards**: All reward chunks. A reward chunk can be modeled by:

```
struct Reward {   // a reward of asset is pending collecting on a pool
    address token;      // the denominator token of the assets
    uint    token_chain // the chain that hosts the denominator token
    uint    amount;     // the asset amount in the denominator token
    uint    pool_id;    // the local/global pool ID
    uint    usdt_equ;   // the usdt-equivalent of the asset amount
}
```

- **Treasury**: Assets reserved for system operation, development, and management. Treasury consists of TreasuryItems.

```
struct TreasuryItem {
    address token;      // the denominator token of the assets
    uint    token_chain // the chain that hosts the denominator token
    uint    amount;     // the amount of the assets in the denominator token
    uint    source;     // the source of treasury item, like performance fee, etc.
    uint    usdt_equ;   // the usdt-equivalent of the asset amount
}
```

- If the system does **Optimize asset/request state**, it changes the states to earn optimal staking reward.

1.2. Design decisions

We choose the **Toggle-Between-Optimize-and_Stay** model for the overall system behavior.

- The system will not always be transitioning, but staying most of the time accepting deposit/withdrawal requests from users.
- If the system **accept**s a deposit request, it
  - collects the asset the user wants to deposit, on the asset's chain,
  - tells the user to wait until the next optimization round, when the system will send some mLP tokens to the user in return for the asset,
  - and book the request with the system for later processing.
- If the system **accept**s a withdrawal request, it
  - collects the mLP tokens the user wants to return, on the mLP's chain,

    - tells the user to wait until the next optimization round, when the system will send some assets of the requested token type in return for the mLP tokens,
    - and book the request with the system for later processing.
- The system will **optimize system asset/request state**, or **transition to new staking** at regular or irregular intervals. The frequency of optimization rounds will be optimized, as frequent moves of asset may incur more costs while infrequent optimization rounds will hinder from quick maneuver of staking.
- When an optimization round is requested, the system leaves the **Staying** state and enters the **Optimizing** state.
- When entering the **Optimizing** state, or an **Optimization round**, the system takes a **system asset/request snapshot**. Then the system transforms/changes the state to an optimal **system asset state** for more rewards, while continuing to **accept** deposit/withdrawal requests, which will be handled at the next round of optimization.
- When an optimization round is finished, the system leaves the **Optimizing** state and enters the **Staying** state.
- In the **Staying** state, the system does nothing but continues to **accept** deposit/withdrawal requests, which will be handled at the next round of optimization.

1.3. Visual description

The **Toggle-Between-Optimize-and_Stay** model of behavior can be expressed in a UML State Machine diagram shown below:

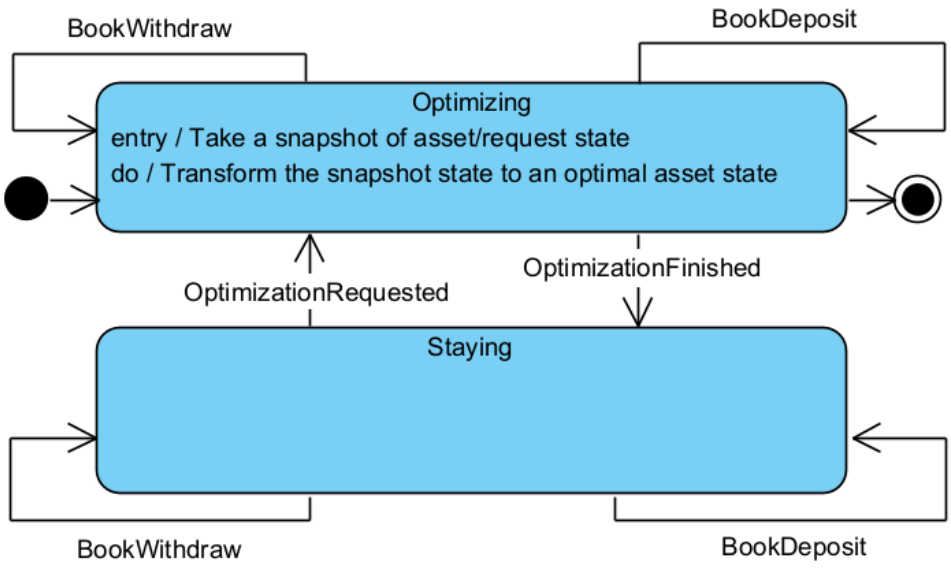

Figure. High-level state machine of Mozaic system

2. Local vaults

We need a module that implements the use case **Control asset move** identified in the requirements specification, solely and completely. We call the module the vault, because from users' perspective:

- the module keeps users' assets, control and logs moves of the assets and profits generated from the assets, and returns the assets with profits.
- no modules other than that module have privilege to carry out these tasks.

2.1. Vaults as smart contracts

According to the requirements, vaults are responsible to, exclusively and at the decentralization level,

- make
- log

all changes to **asset/request state**,

The only way is to deploy smart contracts on chains that cooperate with each other to form an omnichain vault module. We call them local vaults or vault contracts individaully, while they collectively form an omnichain vault.

2.2. Limited responsibility

According to the requirements, vaults do *not* have to

- find the *best possible* asset state to **optimize asset/request state** to
- execute strictly asset move requests coming from outside, like transitionPlan identified in requirements, because there will not be negative profit.

2.3. Local transportation to/from wallets

Local vaults are responsible to:

- Pull assets from the user if a deposit request involves a home token
- Export a deposit request if it involves an away mLP token
- Import an exported deposit request if it involves a home mLP token
- Push mLP tokens to the user if a deposit request involves the home mLP
- Pull mLP to the user if a withdrawal request involves the home mLP token
- Export a withdrawal request if it involves an away token
- Import an exported withdrawal request if it involves a home token
- Push asset to the user if a withdrawal request involves a home token

- Swap at local Dex pools

See the **Reference source code** section for more clarity.

2.4. Local moves of Staking Stock

Local vaults are responsible to:

- Stake assets to local staking pools
- Un-stake assets from local staking pools
- Collect rewards from local staking pools
- Swap at local Dex pools

3. Omnichain staking

3.1. Definition

Omnichain staking requires that:

- Assets can be deposited in any listed token format on any listed chain, *all of users' choice*.
- Deposited assets can be swapped/transferred, and staked in any staking pool on any listed chain, *guided by the system's optimization plan*.
- Staked assets and rewards can be withdrawn in any listed token format on any listed chian, *all of users' choice*.
- Rewards collected can be swapped/transferred, and staked in any staking pool on any listed chain, *guided by the system's optimization plan.*

For a given chain, we define the followings:

$$ChainAssetPlaces = \{ChainVaultWallet\} \cup ChainStakingPools \cup ChainDepositWallets \cup ChainWithdrawalWallets$$

- $ChainVaultWallet$: the vault's wallet on the given chain. They hold from time to time:
  - **Deposits** assets that are pending staking
  - other assets that are transient during optimization
- $ChainStakingPools$: all staking pools on the given chain. They hold:
  - **Stakes** assets,
  - **Rewards** assets, pending collecting
- $ChainDepositWallets$: wallets of all users whose deposit requests are booked and who will receive mLP tokens on the given chain.

    - They **will** receive an amount of mLP sent to the users - the system has to calculate the amount based on the mLP token amount redeemed from the user.
    - The amount to receive will be denoted as a *negative number*.
    - The undefined number will be denoted as **undefined**.
- $ChainWithdrawalWallets$: wallets of all users whose withdrawal requests are booked and who will receive withdrawal assets on the given chain.
    - They **will** receive an amount of asset to sent to the users - the system has to calculate the amount based on the mLP token amount redeemed from the user.
    - The amount to receive will be denoted as a *negative number*.
    - The undefined number will be denoted as **undefined**.

$AllVaultWallets$: the set of $ChainVaultWallet$ of all listed chains.

$AllStakingPools$: the sum of $ChainStakingPools$ of all listed chains.

$AllDepositWallets$: the sum of $ChainDepositWallets$ of all listed chains.

$AllWithdrawalWallets$: the sum of $ChainWithdrawalWallets$ of all listed chains.

$$AllAssetPlaces = AllVaultWallets \cup AllStakingPools \cup AllDepositWallets \cup AllWithdrawalWallets$$

We know:

- an asset move is the move of assets, whether it changes the token type or not, between any the same or different two of $AllAssetPlaces$.
- all asset places have an amount of assets, whether the amount be positive, negative, or **undefined**.

We classify asset moves into the following two groups:

- **Inter-Chain Moves** of a given chain: asset moves that take place between any two of $ChianAssetPlacse$ of the given chain
- **Intra-Chain Moves**: asset moves that take place between one of a chain's $ChainAssetPlaces$ and another of another chain's $ChainAssetPlaces$

3.2. Regular asset move plans

An asset move plan is a set of elementary asset move instructions. We need to eliminate redundant value flows from asset move plans to save the cost of executing the plan.
A regular asset move plan as a plan that has no redundant value flows. **For any asset move plan, there exists a regular equivalent of the original plan. It should be unique(?) and easy to find if we introduce an external asset place as the hub.**

Below comes two example of asset move plan: an irregular plan and its regular equivalent:

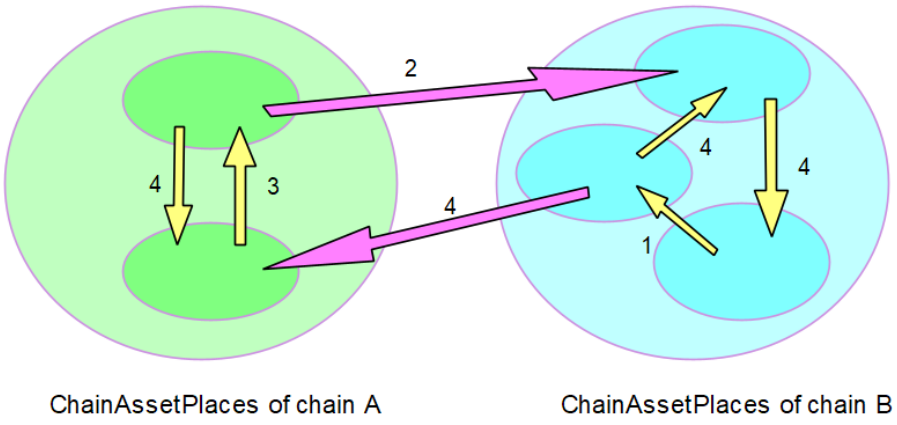

Figure. Irregular asset moves. (swaps are hidden)
Intra-Chain Moves and Inter-Chain Moves have cyclical value moves.
Some asset places both give and take, for example.

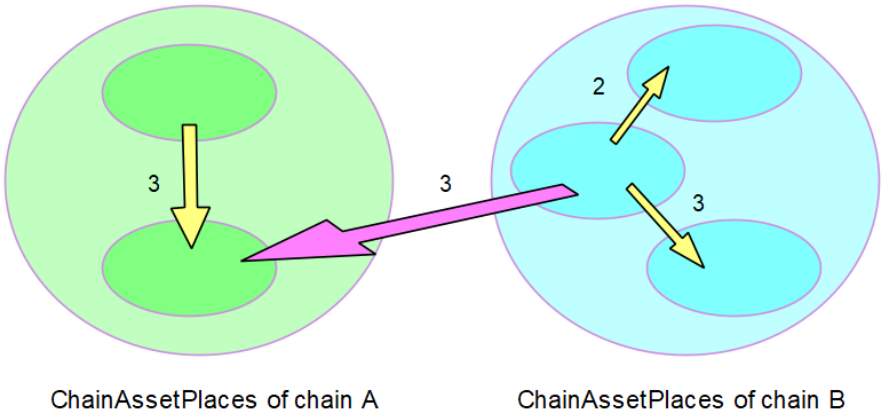

Figure. Regular asset moves. (swaps are hidden)
Intra-Chain Moves and Inter-Chain Moves have no cyclical value moves.
Any group of asset places cannot both take from and give to outside places.

3.3. Design decisions

We deduce the following design decisions:

- Asset moves will be **regularized**. We believe regularization will reduce the total cost of asset moves and the number of inter-chain swap/transfers.
- **ChainValutWallet** will be the hub for regular **Intra-Chain Moves**. This means an **Intra-Chain Move** will be between the **ChainValutWallet** and another of **ChainAssetPlaces**. We wil *not* allow direct asset moves between **ChainAssetPlaces** that are not **ChainVaultWallet**.
- We need one special vault that oversees the cooperation between vaults, including itself, at decentralization level.
  - **Master vault**: the special vault
  - **Master chain**: the chain that hosts the master vault
- The **Master vault** will be the hub for regular **Inter-Chain Moves**. This means an **Inter-Chain Move** will between the **Master vault** and another of local vaults. We wil *not* allow direct asset moves between **ChainAssetPlaces** of different chains.

3.4. Transitioning staking

This algorithm will be executed off-chain to produce a transition plan, because
- it will save huge gas fees that the algorithm would spend if it ran on-chain - the requirements don't require decentralization-level of asset move planning
Note: The off-chain execution of this algorithm raises the concerns of decentralization.

This algorithm handles assets:

- in their USDC-equivalent on the chain, rather than in their own token, when working intra-chain
- in their equivalent of master chains' USDC or the largest giving chains's USDC
- we hope USDC on all chains will have exactly the same price

Algorithm

We define the **asset instances on AssetPlaces** as:
$AssetPlaces^i = \{(T, A, P) \mid place\ P \in AssetPlaces.\ Pplace\ P\ holds\ total\ A\ amount\ of\ T\ token.\ All\ (T, P)\ are\ unique.\}$

Note: This definition is helpful because an asset place, like a wallet or contract, may have different token types of asset.

- Input: Target staking portfolio. (See the coming sections for optimal staking portfolio)
- Output: A transition plan generated and executed in accordance with the input staking portfolio
- Process:
  - Confirm that the target staking portfolio specifies the target asset amount on asset instances.
  - Classify asset instances in $AllAssetPlaces^i$ into giving instances and taking instances
    - If the current asset amount is significantly greater than the target asset amount, it is a **giving asset instance**
      - All of **ChainDepositWallets^i** are all giving asset instances
    - If the current asset amount is significantly less than the target asset amount, it is a **taking asset instance**
      - All of **ChainWithdrawalWallets^i** are all taking asset instances
    - Else, it is a neutral asset instance
    - An asset instance has
      - a positive **giving amount** if it is a giving asset instance, else zero
      - a positive **taking amount** if it is a taking asset instance, else zero
      - a zero giving amount and a zero taking amount if it is a neutral asset instance.
  - Classify chains into giving chains and taking chains
    - If the sum of giving amount of all asset instances in $ChainAsstPlaces^i$ of a given chain is significantly greater than the sum of taking amount, it is a **giving chain**, and the difference is called the **giving amount** of the giving chain.
      - **ChainDepositWallets^i** collectively forms a special giving chain.
      - Be careful not to harm deposits/withdrawals

        - If the sum of taking amount of all asset instances in $ChainAsstPlaces^i$ of a given chain is significantly greater than the sum of giving amount, it is a **taking chain** and the difference is called the **taking amount** of the taking chain.
            - Be careful not to harm deposits/withdrawals
            - **ChainWithdrawalWallets^i** collectively forms a special taking chain.
        - Else, it is a neutral chian
        - A chain has
            - a positive giving amount if it is a giving chain, else zero
            - a positive giving amount if it is a taking chain, else zero
            - a zero giving amount and a zero taking amount if it is a neutral chain
    - Generate a regular **intra-chain asset move plan** for giving chains
        - Collect the local swap prices and fees
        - Collect giving amounts of all giving asset instances to the vault.
        - Swap, divide, and send the collected amount to fill the taking amounts of taking asset instances.
        - Regularize the plan. (See previous sections)
        - The giving amount of this giving chain should remain in the vault.
    - Execute the regular intra-chain asset move plans for giving chains.
    - Generate a regular **inter-chain asset move plan** that divides and transfers the giving amount of assets of giving chains to taking chains.
        - Collect giving amounts of all giving asset chains to the master vault.
        - Swap, divide, and send the collected amount to fill the taking amounts of taking chains.
        - Regularize the plan.
        - There should not remain transient assets in the master vault, except dusts.
    - Execute the regular inter-chain asset move plan.
    - Generate a regular intra-chain asset move plan for taking chains
      (the same as with giving chains)
    - Execute regular intra-chain asset move plans for taking chains.

Note: This algorithm should be tweaked to cope with changing price slippages and fees, and numerical dusts, in implementation phases.

The algorithm is illustrated below:

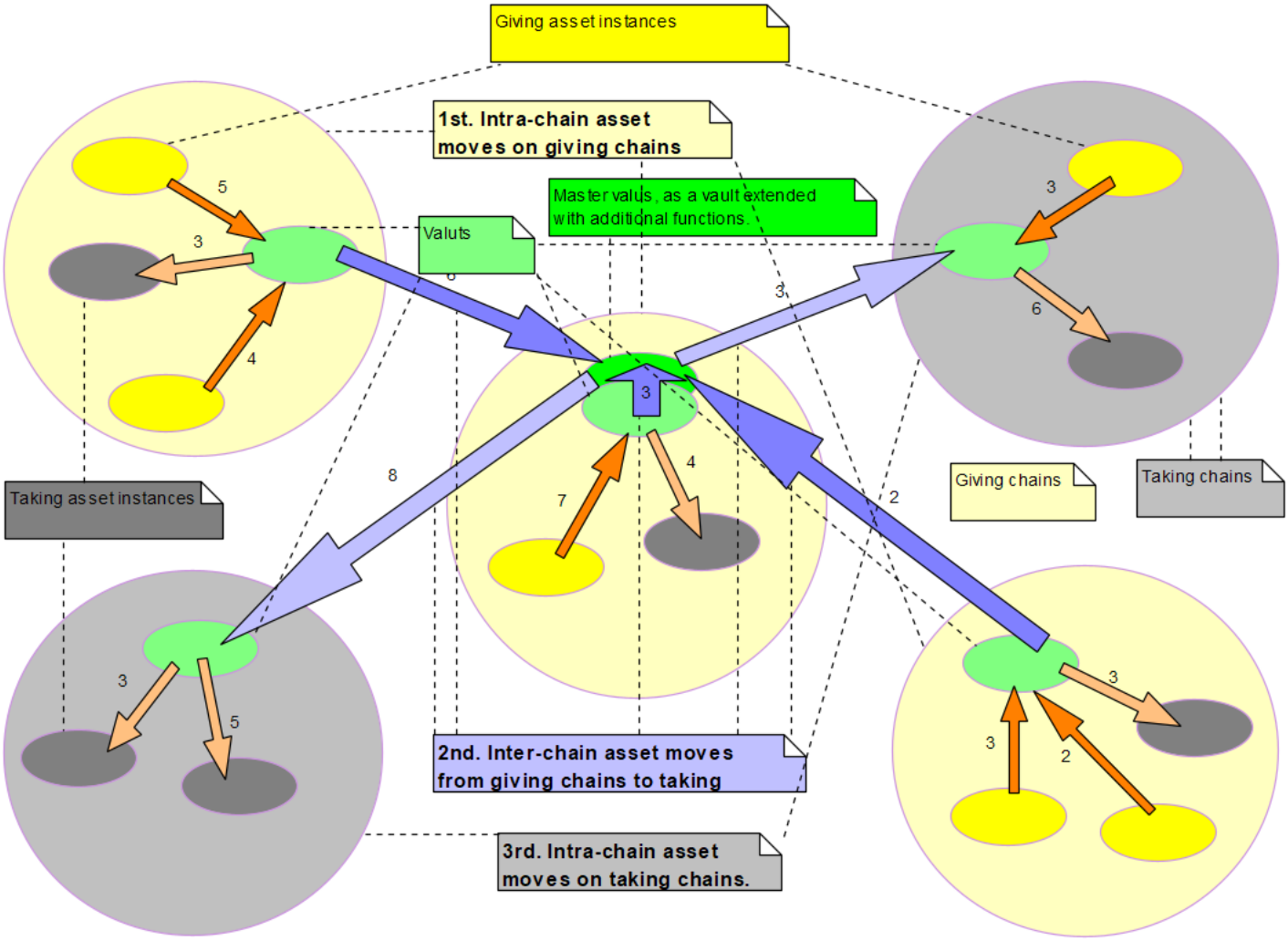

Figure. Transition planning algorithm.
1. Classify asset instances into giving and taking instances. 2. Classify classes into giving and taking chains. 3. (1rd) Finish Intra-chain asset moves between asset instances on giving chains. 4. (2nd) Finish Inter-chain asset moves between vaults from giving chains to taking chains. 5. (3rd) Finish Intra-chain asset moves between asset instances on taking chains. Deposit requests forms a special giving chain, and Withdrawal requests forms a special taking chain.

4. Omnichain mLP token

4.1. Requirements analysis

Omnichain mLP requires that:

- The mLP token should exist on all listed chains.
- All mLP tokens should always have the same global price.
    - When the system **stake**s assets that a user **deposit**ed, the system returns mLP tokens to the user. The amount of the returned mLP token should represent the newly staked asset in the **Staking Stock** immediately after the asset is staked.

    - A user can **withdraw** assets from the **Staking Stock**, in any listed token format on any listed chain, by first returning mLP tokens from their wallet to the system wallet. The amount of asset that is **withdraw**n is the portion of **Staking Stock** that is represented by the returned mLP tokens immediately before the asset is **withdraw**n.

Additional optional use cases the mLP token may have:

- The mLP token cannot have initial supply
- The mLP token can be rebased

4.2. Design decisions

mLP token contracts should be simple and can/should be highly decentralized.

- mLP token contracts will be independent of vaults, except that local vaults should be able to mint and burn local mLP tokens
    - mLP token contracts will be aware of vault contracts
    - mLP tokens will be able to continue to work while vaults are in maintenance mode or being upgraded
- mLP token contracts will be independent of administration
    - mLP tokens will be completely free from administration or DAO
    - mLP tokens will not be upgradeable, for example
- mLP tokens will be inter-chain swapped 1:1 at the level of decentralization
    - LayerZero service will be used
    - (**What if the LZ service gets down?** Make it changeable?)
    - Mint-burn, not lock-release, mechanism will be used
- mLP token swap will be either completely successful or completely reverted on both the source chain and the destination chain
    - It will employ the same technique as Stargate's swap, **if we find no alternatives**.
- **mLP tokens will be promoted with farming**
    - Farming will emit, as reward, portion of the systems profit share
    - This will incentivize more staking
    - Farming will be controlled by administration (Starget's decision models is interesting)

5. Logical components

The overall architectural requiement for vault was/is to **minimize vault as much as possible** leaving most compute to off-chain modules.

The design decisions are as illustrated in the following figure:

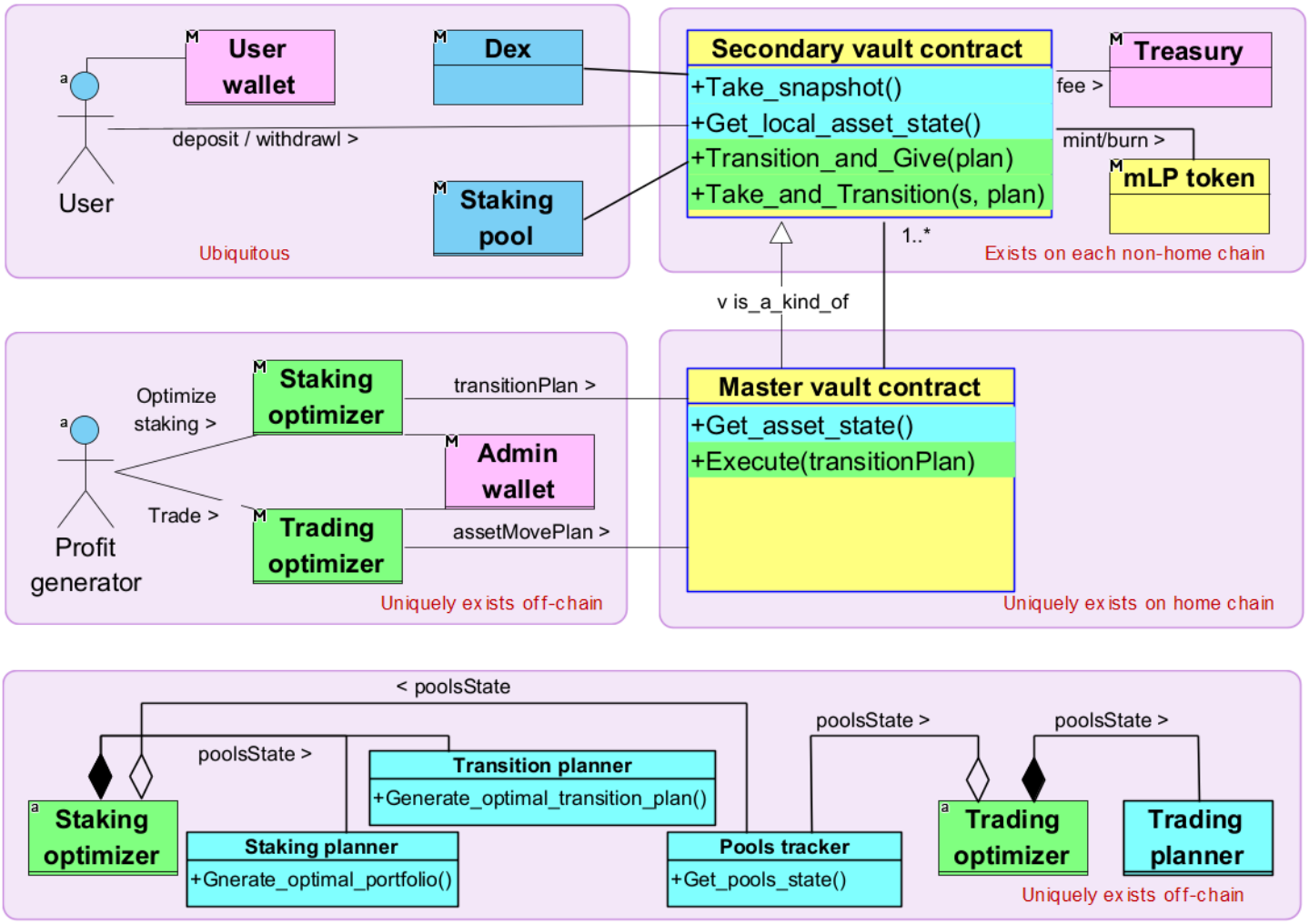

Figure. High-level functional modules of Mozaic DeFi ver. 1.0
Goal: Minimize the functionality of vaults and cross-chain communications.

Functional modules are described below:

- **Secondary vault contract**: This module is a smart contract and deployed on each chain except the home chain. It participates in the cooperation with its peer **Secondary vault contract**s to implement **Control asset move**, This module has at least the following private operations that work locally on its chain: collect_reward(...) and move_staking_asset(...)
- **Master vault contract**: This module is a smart contract, has global operations, in addition to the local operations inherited from **Secondary vault contract**, and is deployed on one and only one of the chains, called **Master chain**, where it takes

the role of **Secondary vault contract**, as well as the the unique role of the master vault operating **mLP token contract**.

- **mLP token contract**: This module manages the mLP token balances of **User**s, which represents their proportional share of the **Staking Stock**.
- **User wallet**: This is a blockchain wallet and identifies a **User**. **User**'s actions; like deposit, withdraw, and harvest; are authenticated/authorized with this wallet.
- **Vault account**: It is the blockchain account of, and controlled by, **Secondary vault contract** and used to store temporary assets, like funds pending staking.
- **Treasury wallet**: This is a blockchain wallet, and a place to store and retrieve system revenues, like fees. It will be better if it is not owned by a human, but be the account of a smart contract that only obeys vault contracts, for better decentralization. It is deployed on all chains.
- **Staking optimizer**: This is an off-chain module that can invoke **Master vault contract**s. This module is globally unique, calculates optimal **transitionPlan**s, and lets the master vault to execute the plans (in cooperation with secondary vaults). *It is an important design decision that the transitionPlan is calculated off-chain, thus leading to transparency and security debates, for the sake of gas- and time- savings:*
    - Transparency debate: **User**s will not be able to track why the system chose particular **transitionPlan**s technically.
    - Security debate: If the calculation of **transitionPlan** is hacked or compromised, then the system will make a less-optimal staking maneuver.
    - Justification: Only the second of the following concerns becomes less transparent, leading to both un-assured best profitability and assured huge gas- and time- savings.
        - how much of what assets from which pool to which pool, is the move about
        - whether all the asset moves are securely and/or reasonably/optimally chosen
        - whether all the asset moves are securely executed and logged
        - whether the move logs are readily available to check later
        - whether the execution of transitionPlan is integrated
- **Trading optimizer**: This off-chain module is similar to **Staking optimizer**, except it relates to trading.
- **Adimin wallet**: This wallet is used to invoke **Master vault contract", in privilege, on behalf of the administrator.
- **Staking planner**: An integral component of **Staking optimizer**, this module predicts the next most profitable **staking_portfolio**, based on **poolsState** provided by **Pools tracker**. Running this module on-chain would enhance transparency, but would at the same time incur huge gas fees and effectively disable the system.
- **Transition planner**: An integral component of **Staking optimizer**, this module predicts the most efficient **transitionPlan**, which is the best procedure of asset move that implements the transitioning to a given **staking_portfolio**, based on the current **poolsState**.
- **Trading optimizer**: This is similar to **Staking optimizer**, except that it relates to trading.
- **Trading planner**: This is similar to **Staking planner**, except that it relates to trading.
- **Pools tracker**: A shared module between **Staking optimizer** and **Trading optimizer**, this module retrieves and tracks all relevant information from chains, like Reward Release Speed, and total Staked mLP of each pool. Running this module on-chain would enhance transparency, but would at the same time incur huge gas fees and effectively disable the system.

6. Omnichain vault

We identify vaults through their surrounding modules interacting with them.
The external actors in the following use case diagram, together with their interactions with vaults are already described above.
We can now explore the use cases of vault.

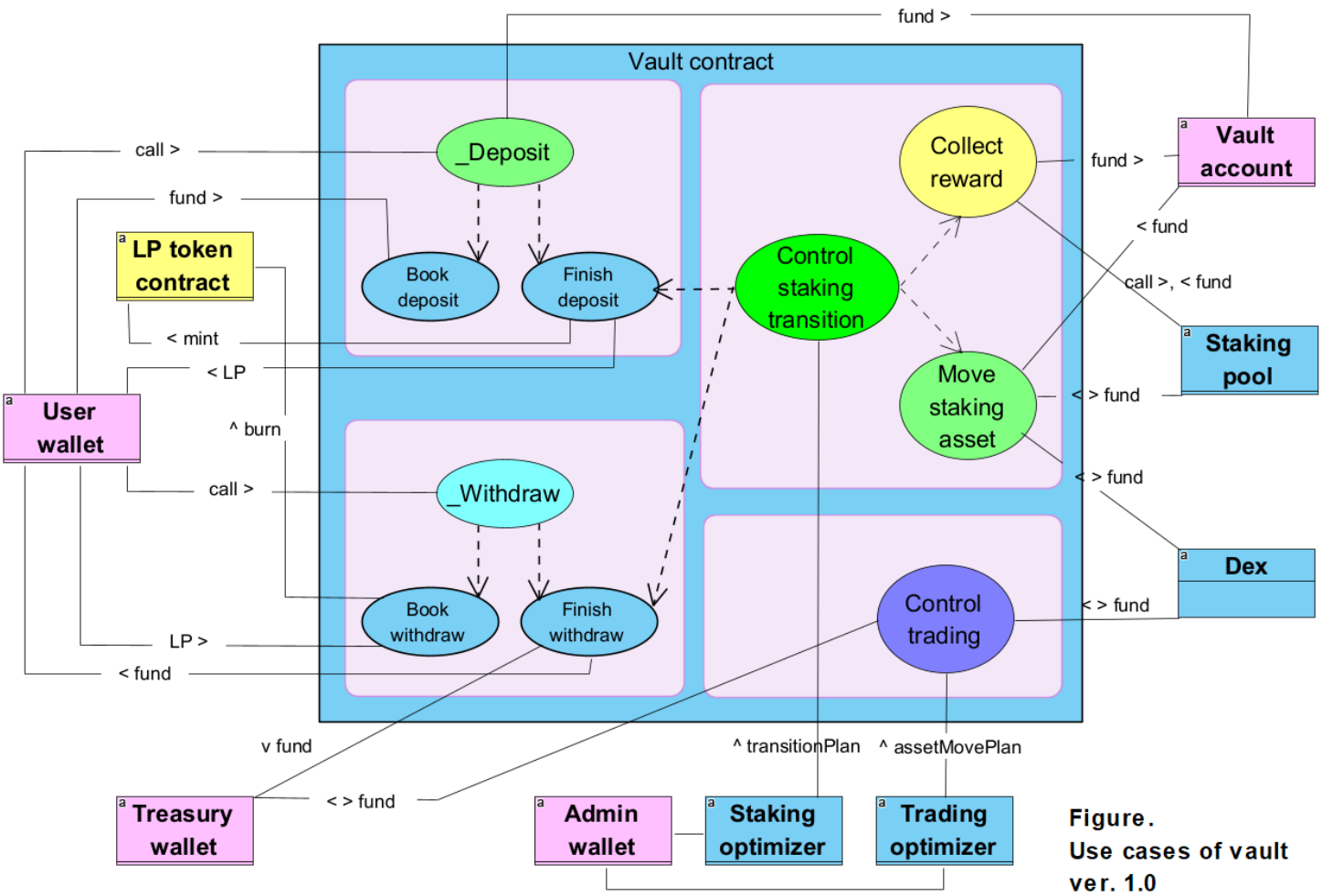

Figure.
Use cases of vault
ver. 1.0

- **_Deposit**: This is what happens at the level of vault contracts when the **Deposit** use case is invoked at the system level. Invoked by the **User wallet** with fund amount to deposit, this use case coordinates the following two use cases.
- **Book deposit**: This use case
  - collects the fund from **User wallet**,
  - books the deposit request with the system,
  - and pauses the session of "_Deposit".
- **Finish deposit**: This use case
  - resume the session of "_Deposit",
  - retrieves the booked deposit request,
  - mints mLP tokens to cover the new fund,

    - returns the mLP tokens to **User wallet**,
    - and helps **Control staking transition** stake the fund.
- **_Withdraw**: This is what happens at the level of vault contracts when the **Withdraw** use case is invoked at the system level. Invoked by **User wallet** with mLP tokens returned, this used case coordinates the following two use cases.
- **Book withdraw**: This use case
    - collects the returned mLP tokens,
    - burns the collected mLP tokens,
    - books the withdrawal request with the system,
    - and pauses the session of "_Deposit".
- **Finish deposit**: This use case
    - resume the session of "_Deposit",
    - retrieves the books withdrawal request,
    - subtract fund, as much as covered by the returned mLP tokens, from the total system assets, and
    - returns the fund to **User wallet**
- **Control staking transition**: This use case transitions to a new asset state by executing **transitionPlan** provided by **Staking optimizer**. (This is the most challenging part of vault implementation.) It does *collectively*, by using **Move staking asset**,
    - **Collect reward**,
    - Collect staked assets, to cover the fund to **Finish withdraw**,
    - **Finish deposit**,
    - **Finish withdraw**,
    - execute remaining part of **transitionPlan**
- **Control trading**: This use case executes **assetMovePlan** provided by **Trading optimizer**.

7. Staking planner

Note. All errors, like numerical processing rounding and price slippage, are ignored at this stage of architectural design.

7.1. Task definition

- Goal: Calculate best staking portfolio, in order to
    - Save vault contracts long calculations of staking optimization, thus to save gas.
    - Keep vault contracts insulated from future algorithm upgrades of staking optimization.
- Consideration
    - Input may not be idealistically consistent inside itself, because an idealistic snapshot of multiple chain states is impossible logically.
    - Output staking portfolio may not be completely/perfectly implemented, because input may have errors and there are unpredictable price slippages sneaked into the calculation.

- Input
    - Deposit requests currently booked
    - Withdrawal requests currently booked

- Current pending rewards
- Current poolsState

- Output
  - optimal staking portfolio

7.2. Definition

- **Pools state**
  $PS = \{PS_i | i = 1..N\}$,
  where PS_i is the i-th pool state:
  $PS_i = (RewardRate_i, RewardToken_i, TotalStake_i, StakingToken_i, MozaicStake_i, Price_i^R, Price_i^S)$
  , where
  - $RewardRate_i$ is the amount of reward emitted during a given time frame
  - $RewardToken_i$ is the token type of reward
  - $TotalStake_i$ is the total amount of staked asset in the pool
  - $StakingToken_i$ is the token type of staked asset
  - $MozaicStake_i$ is the amount of asset staked in the name of Mozaic
  - $Price_i^R$ is the *average* price of reward token in the time frame
  - $Price_i^S$ is the *average* price of staking token in the time frame

  Note: We assume reward on PS_i is calculated as:
  $MozaicReward_i = \frac{RewardRate_i}{TotakStake_i} \times MozaicStake_i$
  If this assumption restricts the applicable staking pools, we have to change the optimization algorithm.

- **Token vectors**
  - **User tokens vector**
    All listed tokens on all listed chains, known to users.
    $UserTokens = \{UserToken_i | i = 1..M\}$
  - **Reward tokens vector**
    $RewardTokens = \{RewardToken_i | i = 1..N\}$
  - **Staking tokens vector**
    $StakingTokens = \{StakingToken_i | i = 1..N\}$
  - **Tokens vector**
    $Tokens = (UserTokens, RewardTokens, StakingTokens)$

- **Asset vectors**
  - **Vector of booked deposit requests**, at transition index t
    $Deposits^t = \{(D_i^t, \ , T_i, \ dLP_i^t) \mid D_i^t : deposited\ amount\ denominated\ by\ token\ T_i, \ dLP_i^t : mLP\ amount, \ all\ at\ transition\ t, i = 1..M\}$
  - **Vector of booked withdrawals requests**, at time index t
    $Withdrawals^t = \{(W_i^t, \ T_i, \ wLP_i^t) \mid D_i^t : withdrawal\ amount\ denominated\ by\ token\ T_i, \ wLP_i^t : mLP\ amount, \ all\ at\ transition\ t, .i = 1..M\}$
  - **Vector of collected rewards**, at transition index t

$Rewards^t = \{(R_i^t,\ T_i) |\ D_i^t : reward\ amount,\ denominated\ by\ \ token\ T_i,\ all\ at\ transition\ t, i = 1..M\}$

- **Vector of staked assets**, at transition index t
  $Stakes^t = \{(S_i^t,\ T_i) |\ D_i^t : stake\ amount,\ denominated\ by\ token\ T_i,\ at\ transition\ t, i = 1..N\}$

- **Transformations**
  - **Transformation $USDT^{+1}$**
    $USDT^{+1}$ transforms an asset vector to USDT-denominated asset vector, by dividing them with relevant USDT prices.
    Example: $USDT^{+}$ transforms (1 USDT, 2 USDC, 3 ETH) to (1, 2.02, 1300), assuming USDT/USDC = 1.01 and USDT/ETH = 1300.
  - **Transformation $USDT^{-1}$**
    $USDT^{-1}$ transforms a USDT-denominated asset vector to tokens-denominated vector, by multiplying them with relevant USDT prices.
    Example: $USDT^{-1}$ transforms (1, 2.02, 1300) to (1 USDT, 2 USDC, 3 ETH), assuming USDT/USDC = 1.01 and USDT/ETH = 1300.
  - **Transformation FOP** - the core of the Archimedes algorithm
    FOP, standing for Find Optimal Portfolio, finds the *best* USDT-denominated vector of staked assets for a given total USDT amount. In other words, *it finds what amounts of (USDT-denominated) value should be allocated to what staking pools, provided that the total (USDT-denominated) value is given*. **best** is relative and subjective.
  - **Transformation Sum**
    Sum sums up all elements of a vector when they are denominated by the same token.

- **Mozaic's asset state**, at transition t
  - Asset snapshot at the beginning of the transition t:
    $MS^t = (Tokens, Deposits^t, -Withdrawals^t, Rewards^t, Stakes^t)$
    Or, simply, $MS^t = (T, D^t, -W^t, R^t, S^t)$
    Note the "-" sign only applies to the asset amount of each element of the vector.
  - Asset snapshot at the end of the transition t:
    $MS^{t+} = (Tokens, 0Deposits, -0Withdrawals, 0Rewards, optimal\ Stakes^t)$
    Or, simply, $MS^{t+} = (T, 0D, -0W, 0R, optimal\ S^t)$
    , where 0D, 0D, and 0R are a vector of zero valued elements of their respective types.

7.3. Formula

If a state transition takes place at time t, Mozaic's asset state MS^t changes to MS^{t+} as shown in the following diagram:

$$
\begin{array}{ccc}
MS^t = (T,\ D^t,\ -W^t,\ R^t,\ S^t) & \xrightarrow{(Resulting\ transition)} & MS^{t+} = (T,\ 0D,\ -0W,\ 0R,\ optimal\ S^t) \\
\Big\downarrow {\scriptstyle USDT^{+1}} & & \Big\uparrow {\scriptstyle USDT^{-1}} \\
MS_U^t = (T,\ USDT^{+1}(D^t),\ ...) & \xrightarrow[(Implicit)]{} & MS_U^{t+} = (T,\ 0D,\ -0W,\ 0R,\ FOP(Total\ in\ USDT)) \\
\Big\downarrow {\scriptstyle Sum} & & \Big\uparrow {\scriptstyle zeros,\ FOP} \\
Total\ in\ USDT & \xrightarrow{Identity} & Total\ in\ USDT
\end{array}
$$

The algorithm for Staking planner MS^t is a chain of transformations:

$$optimial\ S^t = USDT^{-1} \circ FOP \circ Sum \circ USDT^{+1}(T,\ D^t,\ -W^t,\ R^t,\ S^t)$$

- $USDT^{+1}$ and $USDT^{-1}$ are obvious, except that we may need systematic methods to find best Dexes and swap paths.
- FOP is solved analytically, demonstrating about 9% of competitive edge over the public.
- Sum is trivial.
- D^t and W^t can be retrieved from the booked requests of deposits and withdrawals.
- S^t is found when we "Collect reward" pending rewards.

The formula essentially does:

- collect the quantity numbers of all asset amounts available for the new staking:

```
= the **Staking Stock** (i.e. staked assets plus pending rewards),
+ assets received from deposit users,
- assets to send to withdrawing users.
```

- transform them to a single USDT value: Total_in_USDT,
- allocate the Total_in_USDT **optimally** across all staking pools, by using the FOP algorithm, Note: **optimally** is relative and subjective.
- transform the allocated USDT amounts back to the native tokens on the staking pools.
- Send the assets to withdrawing users
- Send mLP tokens to deposit users

8. Inter-chain transportation

8.1. Considerations

- Decentralized inter-chain transportation is required for omnichain-ness
- Decentralized inter-chain transportation may lead to bad User Experience, for its inherent long asynchronous operation
- As such, we need to reduce the use of inter-chain transportation as possible
- Layer Zero is the de facto industry standard of decentralized inter-chain transportation service
- There are total 6 cross-chain calls between the master vault and a (local) vault when the system carries out a round of optimization. (See below diagrams.)
    - If the system is deployed on 10 chains and optimizes 24 times a day, we will have **1,440 cross-chain calls a day**.

    - The 6 cascaded cross-chain calls may pose significant risks to integrity/consistency and User Experience, like runtime responsiveness and coding/maintenance complexity.
- If we compromise on the integrity/consistency of optimization (not on asset moves), by adopting off-chain version of executing transition plans and thus exposing the system to rarely feasible hacking/attacks, then cross-chain calls will be cut down 50%, in return. (See below diagrams.)
- **Not all inter-chain transportation need to be decentralized** (explained below)

8.2. Decentralized operations required

Some vault operations should be decentralized to meet the requirements. Tracking of asset/mLP amount should be executed decentrally, without intermediate off-chain agent or relayer, and with transparent loggs

- Collecting pending rewards from all staking pools to vaults
- Withdrawing from and staking to staking pools to/from vault
- Query for local amounts of assset and mLP token
- Cooperation between vaults to calculate and exchange the information of asset/mLP amounts and indexes derived therefrom
- Sending assets from users' wallets to vaults, on users' deposit requests
- Calculating mLP amount to send to users, in return for their deposited assets
- Sending mLP tokens from vaults to users' wallets, on users' deposit requests
- Sending mLP from users' wallets to vaults, on users' withdrawal requests
- Calculating asset amount to send to users, in return for their returned mLP tokens
- Sending assets from vaults to users' wallets, on users' withdrawal requests

8.3. Employ the LayerZero service

Transparent cross-chain transportation is the fundamental basis of omnichain operations. We choose LayerZero service for our cross-chain transportation.

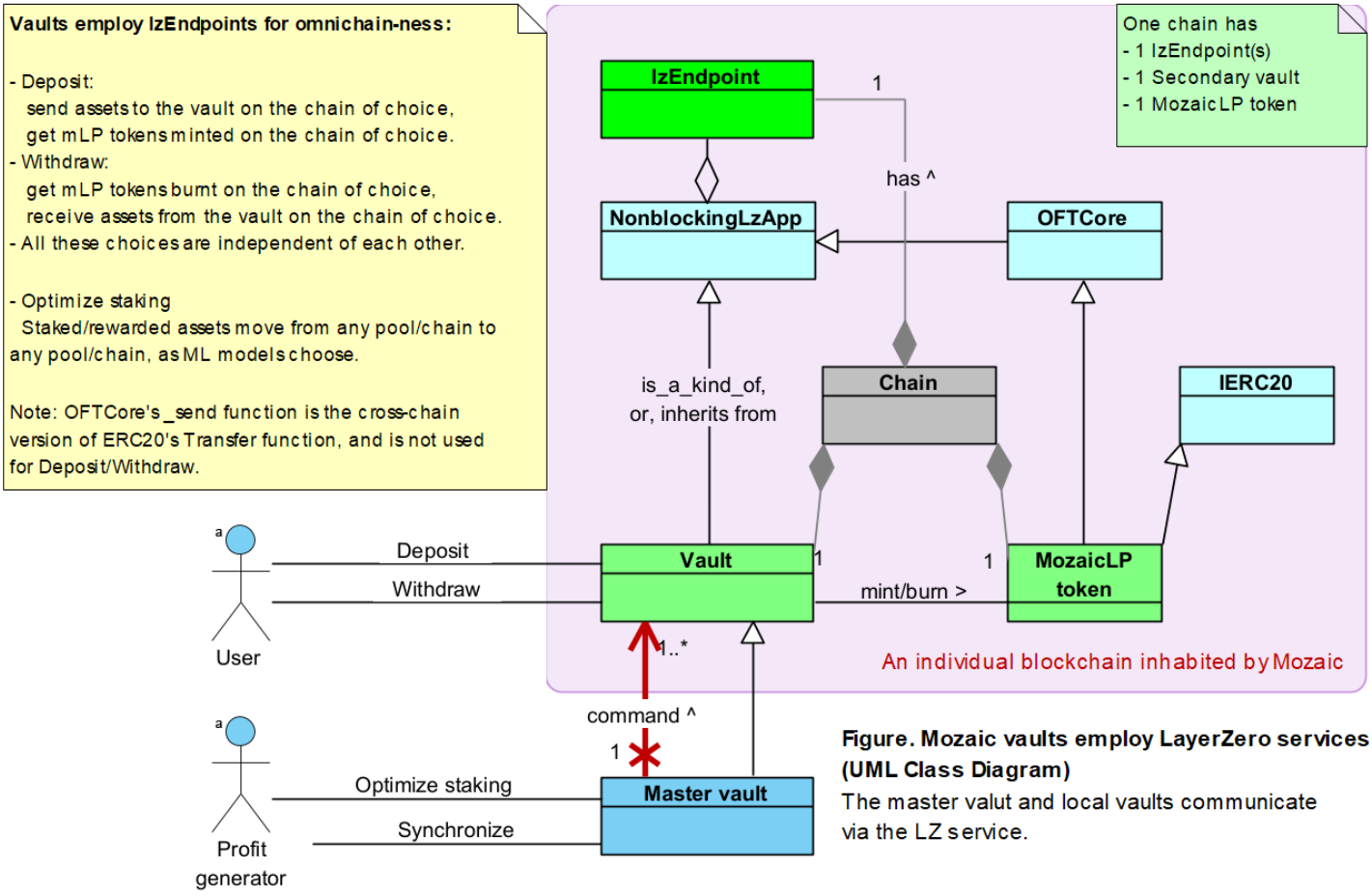

Figure. Mozaic vaults employ LayerZero services (UML Class Diagram)
The master valut and local vaults communicate via the LZ service.

8.4. Operations exemptible of decentralization

Vaults cooperation for staking optimization **does not have to be decentralized**, in the meaning that the optimization doesn't have to provide ideal maximum profit nor have to be successful

- Collecting pools information from chains to off-chain modules, could be done by off-chain modules
- Sending asset move plana to chains, could take detour via Mozaic off-chain modules with Admin wallets, at the risk of
  - the plan could be tempered by (inauditable/unautidited) Mozaic modules or hackers. (But the plan itself is calculated by off-chain modules.)
  - the plan may even fail to be conveyed. (But this type of off-chain failture can also happen when we don't employ off-line detours.)
- Relaying requests between local vaults, during transitioning to a new staking, could take detour via Mozaic off-chain modules with Admin wallets, with the same risks as above

8.5. Design recommendations

- We will choose *decentralized* inter-chain transportation between off-chain modules and vault contracts when finding new optimal staking portfolio and transitioning to the new staking
- Inter-chain messages, once identified as required, will carry as much information as possible.
- Read the section "Reference source code" for more.

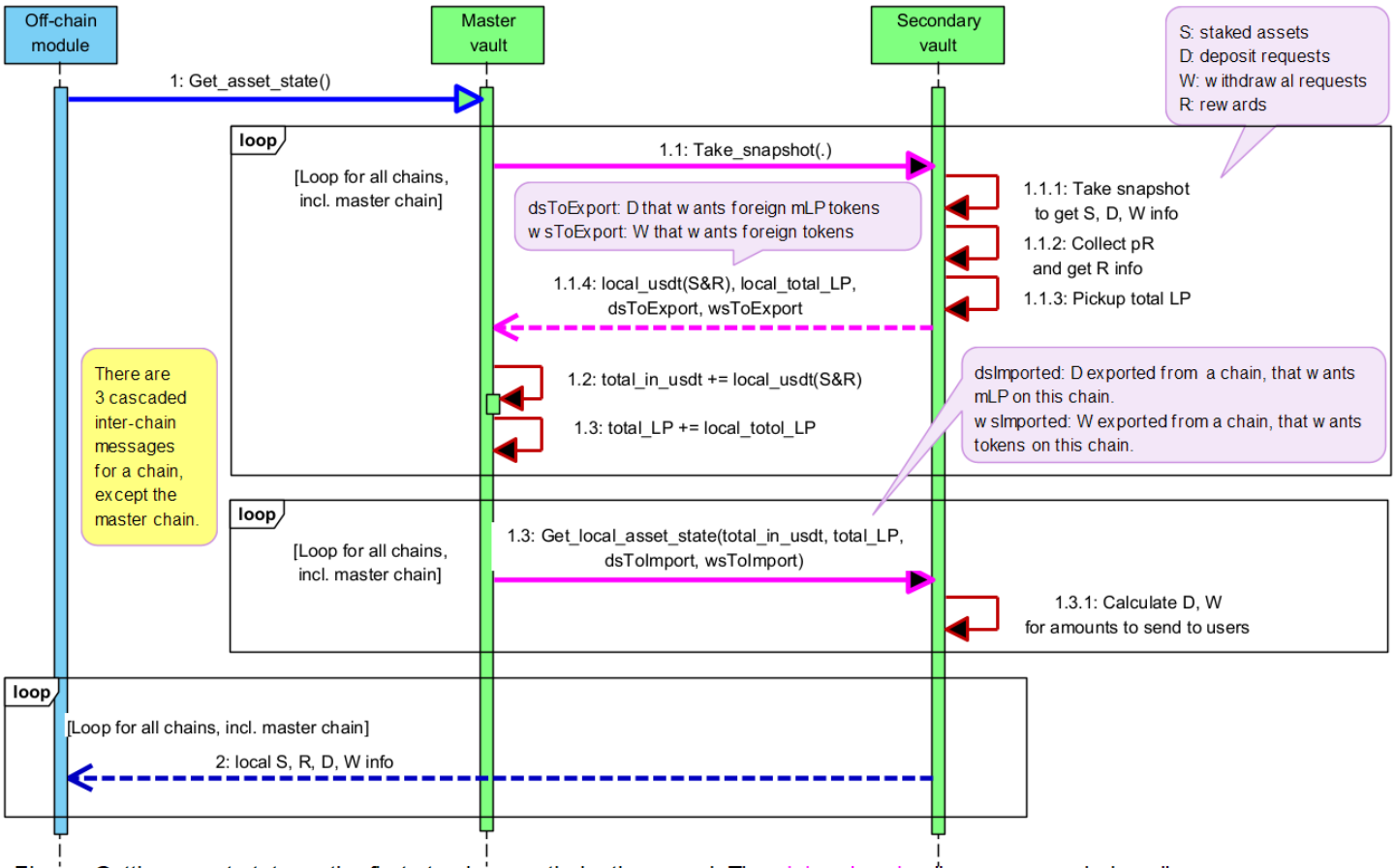

Figure. Getting asset state as the first step in an optimization round. The pink-colored calls are cross-chain calls. The information of S, R, D, and W is created without off-chain involvement. (A contract can call an off-chain module by emitting an event that the module is awaiting.)

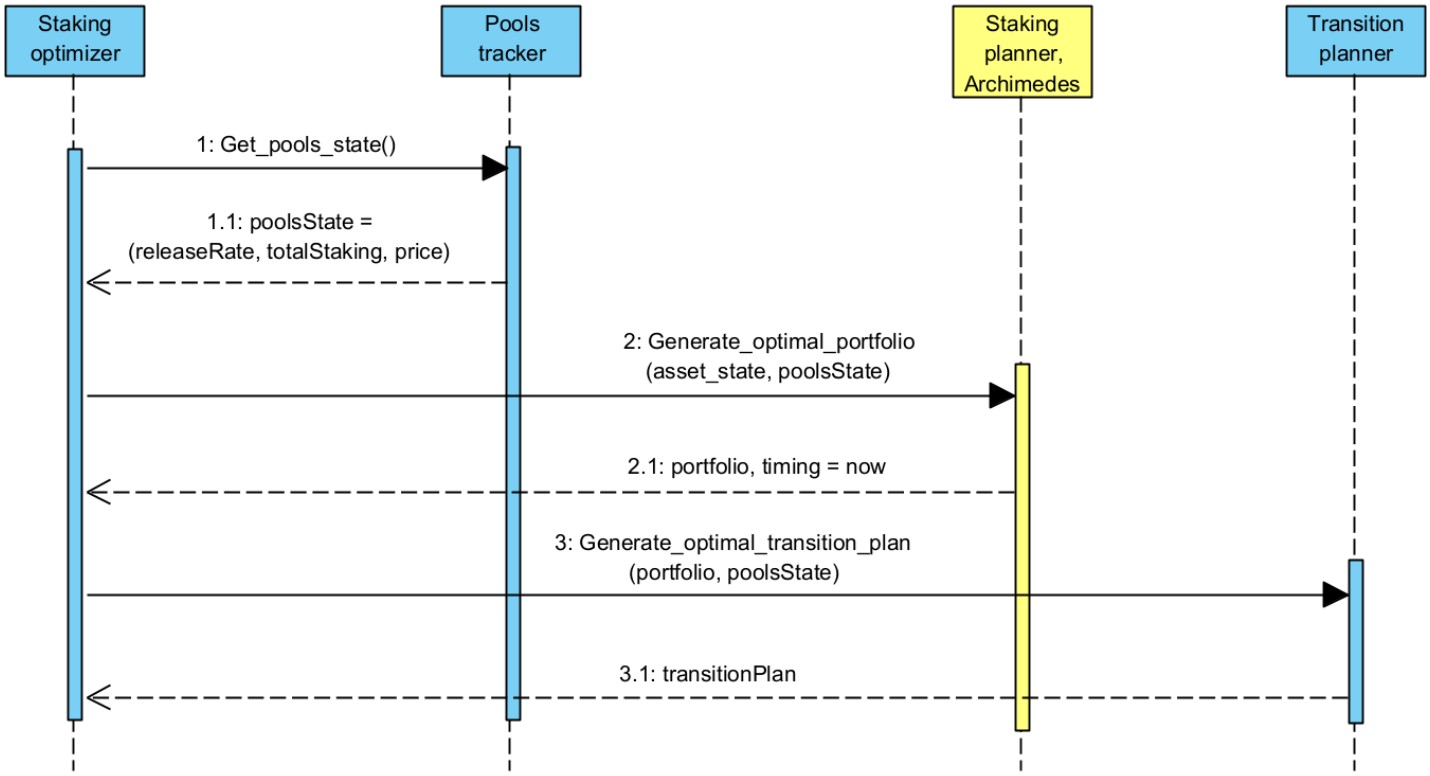

Figure. Generating optimal transition plan, as the second and middle step in an optimization round.
All modules run off-chain. Pools tracker constantly monitors staking pools. Staking planner, the heart of

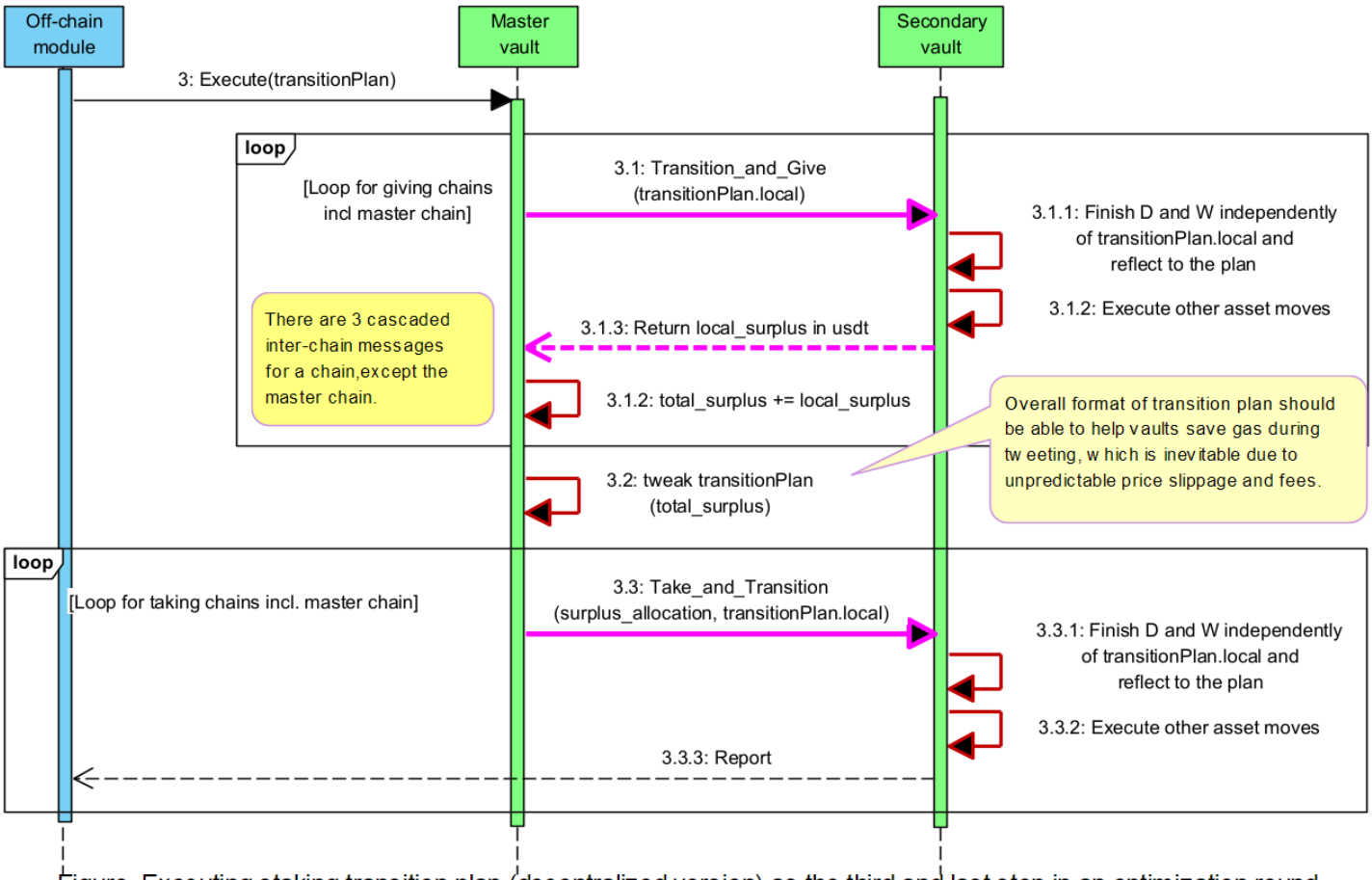

Figure. Executing staking transition plan (decentralized version) as the third and last step in an optimization round. Pink-colored calls are cross-chain calls.

8.6. An off-chain detour for inter-chain transportation

- An off-chain module monitors event logs of a smart contract for a target event happening
- Once detected, the event will be consumed by off-chain modules to produce response
- The produces response will be sent to a proper smart contract

9. Miscellaneous

9.1. Compounding

9.1.1. Considerations

- Rewards should be compounded as frequently as possible, unless the gas fees grows larger than rewards
- Compounding can be executed either:
  - at stocking transition rounds
  - or at its own intervals

9.2. Design recommendations

- Leave it open

9.3. Gas supply

9.3.1. Considerations

- Optimization will spend significant amount of gas, although we minimize vaults
- Gas is spent chain-wise, although the profit generation is not necessarily chain-wise

9.3.2. Design recommendations

- The initial version will be sourcing local gas fees from the local Staking Stock. **If the local Staking Stock is not sufficient for gas fees, the chain will be set inactive**.
- Future versions will maintain a distributed treasure manager to provide local gas spending.

9.4. Auxiliary descriptions of the architecture

9.4.1. Deposit / Withdraw - deposits and rewards mixed 1:1

- We maintain a variable $deposits$ per user.
- Alice's $deposits$ is now 170 stable coins.
- Alice deposits 30 stable coins,
  - Her $deposits$ increases by 30 to become 200.
  - Her total mLP token increases by some amount of mLP token that is calculated to be the share of 30 in the system's resulting renewed Staking Stock.
- When she wants to withdraw with 20 mLP tokens
  - We first find she has a total 100 mLP tokens
  - Decrease her $deposits$ by 200 * 20 / 100 = 40 stable coins
  - Return the withdrawal assets to her, say 120 stable coins, which is a mix of deposits and rewards
  - We know she withdraws 40 principal deposit, together with 120 - 40 = 80 rewards
- **Now, the performance fees = 80 * 10% = 8 stable coins.**
- This means the withdrawal amount is forced to be a 1:1, which is not numerical but proportional, mix of original/principal deposits and rewards generated by staking/compounding them.
- The system will not serve other mix ratios but 1:1, because a ratio is meaning less as $deposits$ and rewards are all mixed and work together with constant compounding.

10. Reference source code

Below comes reference code that sketches and/or decides the code architecture.

```
pragma solidity ^0.8.0;

// imports
import "../libraries/lzApp/NonblockingLzApp.sol";
import "../libraries/stargate/Router.sol";
import "../libraries/stargate/Pool.sol";
import "./OrderTaker.sol";
import "./MozaicLP.sol";

// libraries
import "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";
import "@openzeppelin/contracts/utils/math/SafeMath.sol";

abstract contract SecondaryVault is OrderTaker, NonblockingLzApp {
    function _usdt(address _token, uint _amount) internal returns (uint usdtEq) {
        // use whatever source of price to get usdt-equivalent of
        // the _amount amount of _token token.
    }

    mapping(address => uint) public userTokens;

    function addUserToken(address _token) external {
        require( _token != address(0), "");
        userTokens[_token] = 1;
    }
    function removeUserToken(address _token) external {
        require( _token != address(0), "");
        userTokens[_token] = 0;
    }

    struct Deposit {
        address user;
        address token;
        uint    amount;
        uint    amountLP;   // undefined initially
    }

    struct DepositImported {
        address user;
        uint    usdtEq;
        uint    amountLP;   // undefined initially
    }

    struct DepositToExport {
        address user;
        uint    usdtEq;
        uint    chainId;
    }

    struct Withdrawal {
        address user;
        address token;
        uint    amount;    // undefined initially
        uint    amountLP;
    }

    struct WithdrawalImported {
        address user;
        address token;
        uint    amount;    // undefined initially
        uint    amountLP;
    }
```

```
struct WithdrawalToExport {
    address user;
    address token;
    uint    amountLP;
    uint    chainId;
}

struct Workspace {
    Deposit[] ds;
    DepositToExport[] dsToExport;
    DepositImported[] dsImported;
    Withdrawal[] ws;
    WithdrawalToExport[] wsToExport;
    WithdrawalImported[] wsImported;
}

Workspace private left;
Workspace private right;
bool public transitioning;  // bool has 8 bits. Can put more to fill 256 bits.

function _getPendingWorkspace() internal view returns (Workspace storage) {
    if (transitioning) {
        return left;
    } else {
        return right;
    }
}

function _getStagedWorkspace() internal view returns (Workspace storage) {
    if (transitioning) {
        return right;
    } else {
        return left;
    }
}

uint public thisChain;

// The caller submits _amount of _token, and wants MLP tokens on _chain.
function addDepositRequest(address _token, uint _amount, uint _chain) external  {
    require(userTokens[_token] != 0 && _amount > 0, "Wrong token/amount");
    _safeTransferFrom(_token, msg.sender, address(this), _amount);

    Workspace storage pending = _getPendingWorkspace();
    if (_chain == thisChain) { // The request is local-token for local-mLP
        pending.ds.push( Deposit(msg.sender, _token, _amount, 0) );
        // 0 for mLP amount to send to the user.
    } else { // The request is local-token for away-mLP
        pending.dsToExport.push(DepositToExport(msg.sender, _usdt(_token, _amount), _chain));
        // Foreign chain _chain will store this like:
        // pending.dsImported.push(DepositImported(msg.sender, usdtEq, 0));
        // 0 for the undefined mLP amount to send to the user
    }
}

// The caller submits _amount of mLP, and wants _token tokens on _chain chain.
function addWithdrawalRequest(uint _amountLP, address _token, uint _chain) external  {
    require(userTokens[_token] != 0 && _amountLP > 0, "Wrong token/amount");
    _safeTransferFrom(mLP, msg.sender, address(this), _amountLP);

    Workspace storage pending = _getPendingWorkspace();
    if (_chain == thisChain) { // The request is local-LP for local-token
        pending.ws.push( Withdrawal(msg.sender, _token, 0, _amountLP) );
```

```
            // 0 for the undefined amount of token to send to the user.
        } else { // The request is local-LP for away-token
            pending.wsToExport.push(WithdrawalToExport(msg.sender, _token, _amountLP, _chain));
            // Foreign chain _chain will store this like:
            // pending.wsImported.push(WithdrawalImported(msg.sender, _token, 0, _amountLP));
            // 0 for the undefined amount of token to send to the user
        }
    }
}
```